Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
Université M'hamed Bougara de Boumerdès
Faculté des Sciences, Département : Informatique

TP 1 ► Module : AP	**Année :** S2-Master I2A Pro.	GACEB

L'Objectif de cette première série de TP *est de Configurer un environnement de développement avec PyCharm en installant les bibliothèques qui serviront au deep learning.*

Exercice 1 : Installation de la nouvelle version Pycharm, d'un nouveau projet et configuration de l'environnement de développement

Les séances de TP seront réalisées principalement sur l'IDE PyCharm (sur la nouvelle version : 2025.3) en utilisant des bibliothèques spécialisées pour manipuler les fondamentaux de l'apprentissage profond. Cet IDE (déjà utilisé en premier semestre) est un environnement de développement intégré conçu spécifiquement pour répondre aux besoins des développeurs Python de tous bords (sur Linux, Windows et MacOs). C'est un excellent outil pour le prototypage, facile d'utilisation pour les débutants en développement, tout en étant assez robuste pour satisfaire les exigences des développeurs chevronnés.

La toute première démarche, consiste à télécharger et installer la nouvelle version Pycharm sur votre machine (windows) et création d'un nouveau projet pour les séances de TP selon les étapes suivantes :

1. Télécharger l'installateur depuis la section de téléchargement de PyCharm sur le site de JetBrains (https://download.jetbrains.com/python/pycharm-2025.3.2.1.exe). Lancer l'installateur, cliquer sur Next, choisir l'endroit où vous souhaitez installer le logiciel et cliquer à nouveau sur Next. Suivre l'assistant en cochant toutes les options : comme "Open Folder as Project", l'ajout au PATH, etc.) et cliquer à nouveau sur Next, puis sur Install. PyCharm va maintenant s'installer. Une fois l'installation terminée, cliquer sur Finish.
   A partir de la version 2025.3, JetBrains simplifie son offre en remplaçant l'édition Community par une version unifiée de PyCharm qui regroupe toutes les fonctionnalités sous un seul installateur. Lors du premier mois, l'utilisateur bénéficie d'un accès complet aux outils Professional gratuitement ; à l'issue de cet essai, le logiciel bascule automatiquement vers un mode gratuit permanent qui conserve l'intégralité des options de l'ancienne version Community. Cette mise à jour apporte une amélioration majeure : le support des Jupyter Notebooks, autrefois payant, est désormais accessible gratuitement. Enfin, bien que les étudiants puissent toujours obtenir une licence Professional gratuite via JetBrains, la version community sera donc suffisante pour réaliser les différents travaux pratiques de ce module.
2. La seconde étape consiste à créer votre premier projet sur PyCharm :
   a) Lancer PyCharm sur votre ordinateur, créer un nouveau projet : Sur l'écran de bienvenue ou à partir du menu, cliquer sur "New Project" (Nouveau projet).
   b) Une nouvelle fenêtre de configuration s'ouvre : Dans le champ "Location" (Emplacement), spécifier le chemin où le projet sera enregistré et donner un nom à votre projet (Dans votre cas TP_AP_Nom_Pénom_Groupe).
   c) Version de Python : Assurez-vous que la version de Python sélectionnée dans les options de Virtualenv est bien celle que vous souhaitez utiliser.
   d) Lancer la création : Cliquez sur le bouton "Create" pour générer le projet. PyCharm va créer le dossier du projet et l'environnement virtuel.
   e) A partir du menu ou en cliquant droite sur le nom de projet (menu contextuel), cliquer sur « New » → « Python File », puis saisir dans Name « TP1_Exo1 » et Entrer. Le fichier TP1_Exo1.py est à présent ajouté à votre projet. Une fois le projet créé, vous pouvez commencer à écrire votre code Python dans le nouveau fichier créé.
   f) Installer les bibliothèques
3. La troisième étape consiste à installer les bibliothèques nécessaires pour faire le deep learning en Python.
   - *Les deux bibliothèques les plus utilisées dans ce domaine sont incontestablement PyTorch (Meta/Facebook) et TensorFlow (Google). PyTorch est actuellement la bibliothèque favorite*

de la recherche et du monde académique, permettant de comprendre en profondeur la structure de vos modèles avec une syntaxe intuitive. Son code ressemble à du Python classique et utilise des graphes de calcul dynamiques, ce qui rend le débogage beaucoup plus simple. Cette bibliothèque qui sera utilisée majoritairement en TP est idéale pour le prototypage rapide, les projets innovants et l'apprentissage. En revanche, TensorFlow (via Keras) est la référence dominante dans l'industrie et le déploiement en production, permettant de construire des modèles fonctionnels très rapidement avec un minimum de lignes de code (il est intéressant de le découvrir aussi chez vous).

- *Apprendre le Deep Learning avec PyTorch dans PyCharm est un excellent choix pour combiner la flexibilité de la librairie de Meta avec la puissance d'un environnement de développement professionnel.*

- Pour commencer, vous devez lier PyTorch à votre projet PyCharm. Ouvrez le terminal intégré à PyCharm et exécutez la commande adaptée à votre configuration (CPU ou GPU) disponible sur le sélecteur officiel PyTorch :

```
> pip install torch torchvision torchaudio torchtext matplotlib numpy opencv-python
```

- Pour plus d'informations sur l'installation, veuillez consulter le guide de démarrage sur le site web officiel de PyTorch (https://pytorch.org/get-started/locally).
- Tapez dans le fichier TP1_Exo1.py les instructions suivantes pour confirmer que tout fonctionne :

```
> import torch; print(torch.__version__)
```

4. Un notebook (Jupyter ou Google Colab) est un document interactif qui mélange trois choses dans un seul fichier. Du code exécutable : pour écrire du Python dans des "cellules" et voir le résultat immédiatement en dessous. Du texte riche : pour ajouter des explications, des titres et des images (en utilisant le format Markdown). Des visualisations : Les graphiques et tableaux s'affichent directement dans le document.
   - Pour installer et manipuler vos fichiers .ipynb (notebook) en local, lancer dans le terminal:

```
> pip install jupyter
```

   - Créer un nouveau notebook TP1_Exo1.ipynb, et executer :

```
> import torch; print(torch.__version__)
```

   - Pour chaque exercice de TP1 créer un script python « .py » ou un notebook « .ipynb » sous le nom : `TP1_ExoN.py ou TP1_ExoN.ipynb`
5. **Lier PyCharm à Google Colab et profiter un accès gratuit à un GPU colab :** Depuis la version 2025.3.2, PyCharm offre un support natif pour Google Colab, vous permettant d'exécuter vos notebooks directement sur les serveurs de Google sans quitter votre IDE (vous devez avoir déjà un compte Google pour l'accès à Colab). Étapes à suivre pour lier PyCharm à Google Colab :
   a. **Ouvrir un Notebook** : Dans PyCharm, ouvrez votre fichier .ipynb ou créez-en un nouveau.
   b. Se connecter au compte Google :
   - Dans la barre d'outils du notebook (en haut à droite), cliquez sur la liste déroulante des serveurs.
   - Sélectionnez "Sign In to Google Account" et suivez les instructions dans votre navigateur pour autoriser la connexion.

   c. Créer le serveur Colab :
   - Recliquez sur la liste des serveurs et choisissez "New Colab Server".
   - Donnez un nom à votre serveur et sélectionnez le type d'accélérateur (GPU) pour profiter des ressources gratuites (souvent le T4 par défaut sur le plan gratuit).

   d. **Vérifiez l'activation de GPU :** Une fois connecté, PyCharm agit comme une interface pour l'instance Colab. Vous pouvez vérifier l'allocation des ressources directement dans l'éditeur. Exécutez cette cellule dans PyCharm pour confirmer que le GPU est bien détecté par votre session :

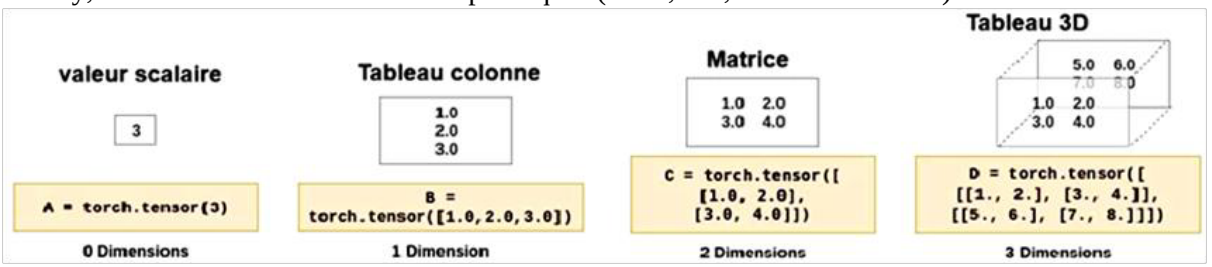
```
import torch ; print(torch.cuda.is_available())
```

Une session peut durer jusqu'à 12 heures. En pratique, pour un usage gratuit intense, vous obtenez souvent entre 2 et 6 heures de calcul GPU effectif par jour avant d'atteindre une limite d'usage. Si vous faites juste de l'édition de texte ou du nettoyage de données léger, passez en mode

CPU dans les réglages de votre serveur Colab sur PyCharm pour économiser vos précieux quotas GPU. Sauvegardez souvent : Montez votre Google Drive dans votre notebook pour que vos modèles et résultats soient enregistrés même si la session coupe brutalement.

Exercice 2 : Manipulation des Tenseurs Pytorch dans : TP1_Exo2.py ou TP1_Exo2.ipynb

Les tenseurs PyTorch (torch.tensor) sont des structures de données multidimensionnelles fondamentales, similaires aux tableaux NumPy, mais optimisées pour le calcul sur GPU et la différenciation automatique dans le deep learning. Ils stockent des données (entrées/sorties) et les paramètres de modèle (poids/gradients), permettant des opérations rapides. Une valeur scalaire est représentée par un tenseur à 0 dimension. De même, un tableau colonne/ligne est représenté par un tenseur 1D, et ainsi de suite. Les tensors peuvent être créés à partir de listes Python, de tableaux NumPy, ou initialisés avec des valeurs spécifiques (zéros, uns, valeurs aléatoires).



- Création et Initialisation des Tenseurs Pytorch : saisir et exécuter les instructions suivantes

```
import torch
# Create a 1D Tensor with just ones in a column --> tensor([1., 1., 1., 1., 1.])
a = torch.ones(5); print(f"1D Tensor with ones : {a}")

# Create a 1D Tensor with just zeros in a column --> tensor([0., 0., 0., 0., 0.])
b = torch.zeros(5) ; print(f"1D Tensor with zeros :{b}")

# Create 1D Tensor with custom values --> tensor([1., 2., 3., 4., 5.])
d = torch.tensor([1.0, 2.0, 3.0, 4.0, 5.0]) ; print(d)

# create 2D tensors
e = torch.zeros(3,2); print(e)
f = torch.ones(3,2) ; print(f)
g = torch.tensor([[1.0, 2.0],[3.0, 4.0]]); print(g)

# Create a 2D Tensor with random values following a normal distribution.
c=torch.randn((2, 2))  ; print(c)

# create 3D tensors
h = torch.tensor([[[1., 2.], [3., 4.]], [[5., 6.], [7., 8.]]]) ; print(h)

#Find out the shape of a Tensor using .shape method,
print( h.shape) ; print(g.shape); print(d.shape)

# Access to element of index 2 in 1D Tensor, all indices starting from 0
print(d[2])

# All elements of a tensor
print(d[:])  ; print(g[:])
print(f"All elements of a tensor g: {g[:]}")

# for 2D tensor, Get element at row 1, column 0
print(g[1,0])

# We can also use the following
print(g[1][0])
```

```
# Similarly for 3D Tensor
print(h[1,0,0])  ;  print(h[1][0][0])
```

Exercice 3 : Conversion de NumPy en tenseurs et de tenseurs en NumPy

Les tenseurs peuvent être convertis en tableaux NumPy, et les tableaux NumPy en tenseurs.

a) Pour transformer un tableau NumPy en tenseur, on peut utiliser la fonction « torch.from_numpy ». Exécuter et observer le résultat de code suivant :

```
np_arr = np.array([[1, 2], [3, 4]])
tensor = torch.from_numpy(np_arr)

print("Numpy array:", np_arr)
print("PyTorch tensor:", tensor)
```

b) Pour transformer un tenseur PyTorch en un tableau numpy, nous pouvons utiliser la fonction .numpy() sur les tenseurs. Exécuter et observer le résultat de code suivant :

```
tensor = torch.arange(4)
np_arr = tensor.numpy()

print("PyTorch tensor:", tensor)
print("Numpy array:", np_arr)
```

Exercice 4 : Opérations entre les tensors

La plupart des opérations présentes dans NumPy existent également dans PyTorch. Vous trouverez la liste complète des opérations dans la documentation de PyTorch, mais nous nous concentrerons ici sur les plus importantes.

a) L'opération la plus simple consiste à additionner deux tenseurs. Saisir et exécuter les instructions suivantes.

```
x1 = torch.rand(2, 3)
x2 = torch.rand(2, 3)
y = x1 + x2
print("X1", x1)
print("X2", x2)
print("Y", y)
```

b) L'appel à x1 + x2 crée un nouveau tenseur contenant la somme des deux entrées. Cependant, il est également possible d'utiliser des opérations sur place, appliquées directement à la mémoire d'un tenseur. On modifie ainsi les valeurs de x2 sans avoir à y accéder à nouveau avant l'opération. Un exemple est présenté ci-dessous :

```
x1 = torch.rand(2, 3); x2 = torch.rand(2, 3)
print("X1 (before)", x1); print("X2 (before)", x2)

x2.add_(x1)
print("X1 (after)", x1); print("X2 (after)", x2)
```

c) Les opérations sur n éléments sont généralement précédées d'un tiret bas (par exemple, « add_ » au lieu de « add »). Une autre opération courante permet de modifier la forme d'un tenseur. Un tenseur de taille (2,3) peut être réorganisé en n'importe quelle autre forme avec le même nombre d'éléments (par exemple, un tenseur de taille (6), ou (3,2), etc.). Dans PyTorch, cette opération est appelée « view ». Executer et oserver le résultats des isntructions suivantes :

```
x = torch.arange(6) ; print("X", x)
x = x.view(2, 3)
print("X", x)
x = x.permute(1, 0) # Swapping dimension 0 and 1
print("X", x)
```

d) Parmi les autres opérations couramment utilisées figurent les multiplications matricielles, essentielles pour les réseaux de neurones. On dispose souvent d'un vecteur d'entrée transformé à l'aide d'une matrice de poids apprise. Il existe plusieurs méthodes et fonctions pour effectuer une multiplication matricielle, dont certaines sont listées ci-dessous :

- torch.matmul : Effectue le produit matriciel de deux tenseurs. Son comportement précis dépend de leurs dimensions. Si les deux entrées sont des matrices (tenseurs bidimensionnels), elle effectue le produit matriciel standard. Pour les entrées de dimension supérieure, la fonction prend en charge le broadcasting (voir la documentation sur le site de Pytorch pour plus de détails). Elle peut également s'écrire sous la forme a @ b, comme avec NumPy.
- torch.mm : Effectue le produit matriciel de deux matrices, mais ne prend pas en charge le broadcasting (voir la documentation).
- torch.bmm : Effectue le produit matriciel en prenant en charge la dimension de lot. Si le premier tenseur T est de forme (bxnxm) et le second R de forme (Bxmxp), la sortie O est de forme (bxnxp) et a été calculée en effectuant b multiplications matricielles des sous-matrices de T et R: Oi=Ti@Ri
- torch.einsum : Effectue des multiplications matricielles et d'autres opérations (par exemple, des sommes de produits) en utilisant la convention de sommation d'Einstein.

Nous pouvons essayer une multiplication matricielle avec torch.matmul ci-dessous.

```
x = torch.arange(6) ;  x = x.view(2, 3) ;  print("X", x)
W = torch.arange(9).view(3, 3) # We can also stack multiple operations in a single line
print("W", W)
h = torch.matmul(x, W) # Verify the result by calculating it by hand too!
print("h", h)
```

Exercice 5 : Maîtriser le flux d'images entre OpenCV et PyTorch

Il est souvent nécessaire de traiter des images à la fois avec OpenCV et PyTorch. Cet exercice simule un pipeline réel où l'on doit jongler entre le traitement d'image classique et le Deep Learning.

1) Flux de traitement classique, OpenCV vers PyTorch
   a) Chargez une image de votre choix en utilisant OpenCV.
   b) Les modèles de Deep Learning travaillent en RGB, mais OpenCV charge en BGR. Effectuez la conversion BGR vers RGB.
   c) Transformez cette image NumPy en un Tensor PyTorch.
   d) Réorganisez les dimensions : passez du format OpenCV (Hauteur, Largeur, Canaux) au format PyTorch (Canaux, Hauteur, Largeur).
2) Flux « Deep Learning Natif », charger une image directement dans un tenseur PyTorch, son affichage et conversion vers une image OpenCV
   a) Chargez une image directement sous forme de Tensor en utilisant uniquement torchvision.
   b) Affichez ce tenseur à l'aide de matplotlib (Attention : vous devrez permuter les axes pour que l'affichage fonctionne).
   c) Convertissez ce Tensor en un objet compatible avec OpenCV (format NumPy, axes HWC, couleurs BGR).
   d) Appliquez un texte sur l'image avec *cv2.putText* et sauvegardez-la.

Exercice 6 : Mise en œuvre de la Descente de Gradient avec PyTorch : Optimisation d'une fonction convexe de degré 2

Pour comprendre la "magie" derrière l'apprentissage automatique et profond. On va utiliser PyTorch pour trouver le minimum de la fonction $f(x)=x^2-4x+4$ avec l'algorithme de la décente de gradient (x0=10, taux d'apprentissage lr = 0.1). Mathématiquement, cette fonction est $(x-2)^2$, donc le minimum est à 2.

1) Initialisation : Déclarez une variable PyTorch x initialisée à *10* Assurez-vous que PyTorch puisse calculer les gradients par rapport à cette variable.
2) Appliquer la descente de gradient classique à l'aide d'une boucle à 50 itérations. À chaque étape :
    a. Calculez la valeur de la fonction $f(x)=x^2-4x+4$
    b. Déclenchez le calcul des gradients (backward).
    c. Mettez à jour *x* en utilisant la formule mathématique de la descente de gradient : $x=x-lr \times gradient$, Afficher l'évolution de *x* chaque *10* itérations.
    Note : Utilisez *with torch.no_grad()*: pour la mise à jour et n'oubliez pas de remettre le gradient à zéro.
3) L'automatisation avec Optimizer : Réinitialisez *x* à *10*, utilisez cette fois l'objet *torch.optim.SGD* pour automatiser la mise à jour à l'aide de la méthode de descente de gradient stochastique.
    a. Remplacez votre formule de *DG classique* par *optimizer.step().*
    b. Remplacez la remise à zéro manuelle par *optimizer.zero_grad().*
4) Application de l'optimiseur *Adam* : Réinitialisez une nouvelle fois x à 10.0.
    a. Au lieu de *SGD*, utilisez l'optimiseur *torch.optim.Adam.*
    b. Gardez le même taux d'apprentissage (*lr=0,1*)
    c. Observez la vitesse de convergence. Est-elle plus lente ou plus rapide que le SGD sur les premières étapes ? Note : Adam utilise des "moments" (une sorte de mémoire des gradients précédents) pour ajuster sa trajectoire. Discuter de la précision de convergence, ajouter des itérations supplémentaires si nécessaire et observer le résultat. Changer la valeur de lr et observer le résultat.
5) Automatisation de la convergence (Critère d'arrêt) : En conditions réelles, on ne sait jamais à l'avance combien d'itérations sont nécessaires. On utilise alors un critère d'arrêt basé sur la précision. Au lieu de fixer un nombre arbitraire d'itérations (comme 50 ou 100), modifiez la boucle pour qu'elle s'arrête automatiquement lorsque le gradient devient "négligeable".
    a. Utilisez une boucle *while* et définissez un seuil de précision (ex: eps=10^{-6}).
    b. La boucle doit continuer tant que la valeur absolue du gradient est supérieure à ce seuil
    c. Affichez le nombre total d'itérations qu'il a fallu à Adam pour converger.
6) Visualisation de la convergence, reprenez l'algorithme de votre choix (SGD ou Adam).
    a. Créez une liste vide nommée history_loss.
    b. À chaque itération de la boucle, ajoutez la valeur actuelle de la *loss* dans cette liste (utilisez *.item()* pour ne garder que le nombre).

c. À la fin de l'entraînement, utilisez *matplotlib.pyplot* pour tracer la courbe représentant l'évolution de la perte en fonction du nombre d'époques.

Exercice 7 : Reconnaissance des chiffres manuscrits avec un réseau MLP sur PyTorch

L'objectif de cet exercice est construire un modèle neuronal capable de reconnaître des chiffres manuscrits (0-9) à l'aide d'un MLP (Multi-Layer Perceptron ou Perceptron Multicouche) sur PyTorch. L'apprentissage de ce modèle repose sur un processus itératif en ajustant ses poids (paramètres) pour minimiser ses erreurs de prédiction. C'est ici qu'intervient la descente de gradient (DG) comme une boussole en guidant l'algorithme vers le point (poids+ biais) où l'erreur est la plus faible possible (le minimum global). En calculant la pente de la fonction de perte (gradient), elle permet de déterminer dans quelle direction et avec quelle intensité modifier les poids et les biais à chaque itération pour faire converger le modèle vers une précision optimale. Sans DG, le réseau de neurones modifierait ses paramètres au hasard, ce qui rendrait l'apprentissage extrêmement lent, voire impossible. Elle permet de passer d'un simple constat d'échec à une optimisation ciblée.

L'apprentissage d'un MLP repose sur les étapes suivantes :

- Préparation d'un dataset d'images représentatives avec des étiquettes des classes des chiffres.
- Initialisation : définition de l'architecture (nombre de couches et de neurones) et on attribue des valeurs aléatoires aux poids (connexions) et aux biais.
- Propagation avant (Forward Propagation) : Les données du dataset passent à travers le réseau. Chaque neurone effectue un calcul mathématique pour transmettre l'information jusqu'à la couche de sortie, qui génère une prédiction.
- Calcul de l'erreur (Loss Function) : On compare la prédiction du modèle avec la valeur réelle attendue. La différence est mesurée par une "fonction de perte".
- Rétropropagation (Backpropagation) : C'est l'étape cruciale. L'erreur est renvoyée en arrière dans le réseau pour identifier quels poids ont le plus contribué à l'imprécision.
- Optimisation (Descente de gradient) : On ajuste légèrement les poids et les biais dans la direction qui réduit l'erreur. On répète ensuite les étapes 2 à 5 (pendant plusieurs époques) jusqu'à ce que le modèle soit performant.

Le dataset MNIST est le standard de référence pour développer ce système de reconnaissance de chiffres manuscrits. Contenu : Chiffres (classes) de 0 à 9 écrits à la main par des lycéens et des employés du bureau du recensement américain avec 70 000 images au total qui sont devisées de la façon suivante : 60 000 images (pour ajuster les poids du modèle durant l'entrainement) et 10 000 images (utilisées pour évaluer et tester la capacité de généralisation de ce modèle sur des nouvelles images de chiffres).

Chaque image est en niveaux de gris de 28x28 =784 pixels.

Figure 1 : Quelques exemples d'images du dataset MNIST

Travail à faire sur un notebook à l'aide de PyTorch :

a) Télécharger le dataset MNIST en utilisant torchvision, concevoir, entraîner et évaluer un réseau de neurones de type MLP sur PyTorch pour classer ces images dans l'une des 10 classes (0 à 9). Vous devrez analyser aussi l'impact de différents hyper-paramètres sur la performance du modèle. La structure de modèle doit être configurée de la façon suivante :

- Couche d'entrée de 784 neurones (correspondant aux pixels aplatis).
- Couche Cachée de 128 neurones avec une fonction d'activation ReLU.
- Couche de sortie : 10 neurones (un par classe/chiffre) avec une fonction de perte CrossEntropy.
- Optimiseur : Adam (tester les autres).

b) Entrainer puis testez le modèle avec un batch size de 64 et 3 époques pour les optimiseurs suivants : *Adam, SGD, RMSprop*. Utiliser l'Accuracy et le F-Score comme métriques d'évaluation.

c) Avec l'optimiseur d'Adam, modifier le batch size d'entrainement à 32, 128 et 256 et le nombre d'époques à 5, 10, 15 et 20 et calculer pour chaque configuration l'Accuracy et le F-score sur le dataset de test. Rapporter vos résultats dans plusieurs tableaux et commenter vos observations.

d) Sauvegarder le modèle résultant sur disque.